Python Objects: Types, Attributes, Methods, and the Dot

MATH 5200 — Computing Using R & Python

Table of contents

1 Purpose **2**

2 Names and Objects **3**
2.1 Two names can refer to the same object . . . 3
2.1.1 Names are not boxes . . . 3

3 Every Object Has a Type **4**

4 Attributes and the Dot **4**

5 Attributes Are Not Necessarily Methods **5**

6 Methods **6**

7 Functions Versus Methods **6**
7.1 Why not `x.len()`? . . . 6

8 Special Methods and Dunder Names **7**
8.1 Why does Python work this way? . . . 7

9 Python Protocols **8**

10 `__str__` and `__repr__` **9**

11 Mutation and Method Return Values **9**

12 Mutable and Immutable Objects **10**

13 Where Do Methods Come From? **10**

14 A Tiny Class Reveals What `self` Means **11**
14.1 What is `self`? . . . 11

15 `__init__` and Object Construction **12**

16 Objects Can Contain Other Objects **12**

17 Modules Are Objects and Namespaces **13**
17.1 Modules provide namespaces . . . 14

18 Packages and Nested Names **14**

19 Namespaces and Scope **15**

20 Four Things That Look Similar but Are Different **16**
20.0.1 `x` . . . 16
20.0.2 `obj.x` . . . 16

20.0.3 `d["x"]` . . . 16

21 Introspection: Investigating an Object **16**
21.1 `type()` . . . 16
21.2 `id()` . . . 16
21.3 `dir()` . . . 16
21.4 `help()` . . . 17
21.5 `callable()` . . . 17

22 Looking at an Object's Attributes **17**

23 `inspect`: Looking More Closely **17**

24 Reading Python Expressions Through the Object Model **18**

25 A General Strategy for Reading Python **18**

26 Worked Investigation **19**
26.0.1 What is `statistics`? . . . 19
26.0.2 What does it contain? . . . 19
26.0.3 What is `mean`? . . . 19
26.0.4 What arguments does it accept? . . . 20
26.0.5 What happens when we call it? . . . 20

27 Capstone: What Is Python Doing? **20**

28 The Big Picture **21**
28.1 Names refer to objects . . . 21
28.2 Objects have types . . . 21
28.3 Objects have attributes . . . 21
28.4 Some attributes are methods . . . 21
28.5 Functions are callable objects . . . 22
28.6 Special methods implement protocols . . . 22
28.7 Objects can be instances of classes . . . 22
28.8 Modules provide namespaces . . . 22
28.9 Python can inspect itself . . . 22

29 The Most Useful Mental Model **22**

30 What You Should Take Away **23**

1 Purpose

Python code can sometimes look like a collection of unrelated rules:

```
x.append(4)
len(x)
x[0]
np.linalg.norm(x)
```

These expressions become much easier to understand once we have a mental model of **objects, names, types, attributes, methods, and protocols**.

The goal of this note is not to teach object-oriented programming. Instead, it explains the computational model underlying ordinary Python code.

By the end, you should be able to answer questions such as:

- What is a Python variable?
- What is an object?
- What does the dot (`.`) mean?

- What is the difference between an attribute, a function, and a method?
- Why do some operations use functions such as `len(x)` rather than methods such as `x.len()`?
- What are names such as `__len__` and `__repr__`?
- What happens when we import a module?
- How can we investigate an unfamiliar object?

A useful high-level picture is:

```
                    Python program
                          |
                    names / bindings
                          |
                    Python objects
                    /             \
                data               behavior
                 |                    |
           attributes              methods
```

2 Names and Objects

Consider:

```
x = [1, 2, 3]
```

It is tempting to imagine that Python creates a box called `x` containing the list. A more useful model is that Python creates a **list object** and binds the name `x` to that object.

We can investigate the object:

```
x = [1, 2, 3]

type(x)
id(x)
```

`type(x)` tells us what kind of object `x` refers to. `id(x)` gives an identity associated with that particular object during its lifetime.

2.1 Two names can refer to the same object

```
x = [1, 2, 3]
y = x

x is y
x == y
```

Both expressions are `True`, but they mean different things:

- `==` asks whether two objects have equal values.
- `is` asks whether they are the same object.

Now mutate the list:

```
y.append(4)

x
y
```

Both names see the change because they refer to the same mutable object.

2.1.1 Names are not boxes

Compare this with:

```
x = 10
y = x

x = 20

x
y
```

Here, rebinding `x` does not change the object to which `y` refers.

The useful mental model is therefore:

> **A name refers to an object. Assignment binds or rebinds a name; it does not mean "put a value into a variable box."**

This distinction becomes especially important for mutable objects.

3 Every Object Has a Type

Python objects have types:

```
type(42)
type(3.14)
type("hello")
type([1, 2, 3])
type({"a": 1})
type(None)
type(True)
```

Common built-in types include:

- `int`
- `float`
- `bool`
- `str`
- `list`
- `tuple`
- `dict`
- `set`

We can test whether an object belongs to a type:

```
x = [1, 2, 3]

isinstance(x, list)
isinstance(x, int)
```

An individual object is often called an **instance** of its type. Thus:

```
list                    → a type
[1, 2, 3]               → an instance of list
```

More precisely, Python's object model is built around classes and instances. A class defines the behavior associated with a type, while an instance is an individual object of that type.

We will use this idea later, but we do not need to study object-oriented programming to benefit from it.

4 Attributes and the Dot

Now consider:

```
x = [1, 2, 3]

x.append
```

The expression does not call anything. It **looks up an attribute** named `append` on the object referenced by `x`.

The dot notation

```
object.attribute
```

means, roughly:

> Find the attribute with this name associated with this object.

For example:

```
x.append
x.copy
x.count
```

We can discover many available names with:

```
dir(x)
```

A useful way to hide Python's special names is:

```
[name for name in dir(x) if not name.startswith("_")]
```

The important idea is:

> **The dot is attribute access. It is not intrinsically a method operation.**

5 Attributes Are Not Necessarily Methods

Consider the `math` module:

```
import math

math.pi
math.sqrt
```

These are both attributes of the `math` module, but they are different kinds of objects.

```
callable(math.pi)
callable(math.sqrt)
```

`math.pi` is a value. `math.sqrt` is callable.

Thus:

```
math.pi
```

retrieves an attribute, while:

```
math.sqrt(16)
```

retrieves an attribute and then calls the resulting object.

A useful decomposition is:

```
math.sqrt(16)
|    |     |
|    |     +-- argument
|    +-------- attribute lookup
+------------- name lookup
```

The parentheses `()` are the **call operator**. The dot `.` is the **attribute-access operator**.

These are separate operations.

6 Methods

Return to the list:

```
x = [1, 2, 3]

x.append
```

The `append` attribute is a method associated with the list object. Calling it:

```
x.append(4)
```

invokes that method.

A useful first approximation is:

> **A method is callable behavior accessed through an object's attributes.**

This gives us the familiar syntax:

```
object.method(arguments)
```

For example:

```
x.append(4)
x.count(2)
x.copy()
```

The object on the left of the dot is the object whose behavior we are using.

7 Functions Versus Methods

Python also has ordinary functions:

```
len(x)
sum(x)
max(x)
sorted(x)
```

These are not methods of `x`.

Compare:

```
len(x)
```

with:

```
x.append(4)
```

The first is a function call. The second is attribute lookup followed by a method call.

You can see this directly:

```
len
x.append
```

Both produce callable objects, but they are obtained differently.

7.1 Why not `x.len()`?

Python uses ordinary functions for many **generic operations** that can work with many different kinds of objects:

```
len(x)
sum(x)
max(x)
sorted(x)
```

For example:

```
len([1, 2, 3])
len("hello")
len({"a": 1, "b": 2})
```

This distinction becomes especially important when we look at Python's special methods.

8 Special Methods and Dunder Names

Names surrounded by double underscores are commonly called **dunder names**, short for "double underscore."

Examples include:

```
__len__
__getitem__
__iter__
__str__
__repr__
__add__
```

These are special methods that participate in Python's object protocols.

For a list:

```
x = [10, 20, 30]

x.__len__()
```

produces the same length as:

```
len(x)
```

Likewise:

```
x.__getitem__(0)
```

corresponds conceptually to:

```
x[0]
```

and:

```
x.__add__([40, 50])
```

corresponds to:

```
x + [40, 50]
```

You normally use the language-level operations:

```
len(x)
x[0]
x + y
```

rather than calling these special methods directly.

8.1 Why does Python work this way?

It lets Python provide a common interface for different object types.

For example, many objects can have a length:

```
len([1, 2, 3])
len("hello")
len((10, 20))
```

Python does not need to know in advance what `x` is. It uses the appropriate protocol for the object.

A useful conceptual model is:

```
len(x)
   |
   v
Python uses the object's length protocol
   |
   v
x.__len__()
```

Similarly:

```
x[0]
  |
  v
Python uses the object's indexing protocol
  |
  v
x.__getitem__(0)
```

The details of Python's data model are more subtle than these simplified diagrams, but this model is sufficient for most ordinary programming.

9 Python Protocols

A **protocol** is a set of operations or methods that allow an object to participate in a particular kind of computation.

Examples include:

Operation	Related protocol
`len(x)`	length
`x[i]`	indexing
`for item in x`	iteration
`x + y`	addition
`str(x)`	string representation
`repr(x)`	developer representation

This is one of the most important ideas in Python:

> **Python's built-in operations often work by asking objects to provide particular behavior.**

This is why different types can work with the same Python syntax.

For example:

```
for x in [1, 2, 3]:
    print(x)

for x in "hello":
    print(x)
```

A list and a string are very different objects, but both support iteration.

10 `__str__` and `__repr__`

Two particularly useful special methods control how objects are represented.

```
x = [1, 2, 3]

str(x)
repr(x)
```

The distinction is subtle but useful:

- `str(x)` aims at a readable representation.
- `repr(x)` aims at a representation useful for debugging and development.

For many built-in objects, the results look similar:

```
str(x)
repr(x)
```

`print()` uses string conversion:

```
print(x)
```

while an interactive Python session often displays an object's representation:

```
x
```

The important conceptual point is that an object can define standard ways for Python to represent it.

11 Mutation and Method Return Values

Methods can mutate an object, return a new object, or return some other result.

Consider:

```
x = [3, 1, 2]

x.sort()
x
```

`sort()` changes the list itself.

Now compare:

```
x = [3, 1, 2]

y = sorted(x)

x
y
```

`sorted()` returns a new sorted object while leaving the original list unchanged.

A common beginner mistake is therefore:

```
x = [3, 1, 2]
y = x.sort()

y
```

`y` is `None` because `list.sort()` performs the mutation in place rather than returning the sorted list.

This distinction is part of understanding an object's interface:

> **Always ask: Does this operation mutate the object, return a new object, or return some other result?**

12 Mutable and Immutable Objects

Some objects can be changed after they are created; others cannot.

Lists are mutable:

```
x = [1, 2, 3]
x.append(4)
x
```

Strings are immutable:

```
x = "hello"

x.upper()
x
```

`upper()` produces a new string; it does not modify the original string.

Similarly:

```
x = (1, 2, 3)
```

cannot be changed in place.

This distinction helps explain the behavior of names that refer to the same object:

```
x = [1, 2]
y = x

y.append(3)

x
```

versus:

```
x = "hello"
y = x

y = y.upper()

x
y
```

The issue is not simply "variables are mutable" or "variables are immutable." **Objects** have these properties.

13 Where Do Methods Come From?

Consider:

```
x = [1, 2, 3]

type(x)
list.append
```

`list` is the type of `x`, and `append` is behavior defined for list objects.

You can inspect the type itself:

```
dir(list)
```

and:

```
list.append
list.count
list.sort
```

The same pattern appears for other built-in types:

```
str.upper
dict.keys
set.add
```

A useful conceptual picture is:

```
                          list
                           |
              defines list behavior
                           |
              +------------+------------+
              |            |            |
           append        sort         count
              |
              v
      [1, 2, 3]      <- an instance of list
```

This is where the vocabulary of **class** and **instance** becomes useful:

- `list` is a class/type.
- `[1, 2, 3]` is an instance of `list`.
- `append` is behavior defined for list objects.

14 A Tiny Class Reveals What `self` Means

We do not need to learn object-oriented programming to understand method syntax.

Consider this small class:

```
class Counter:
    def __init__(self, value=0):
        self.value = value

    def increment(self):
        self.value += 1
```

Create an instance:

```
c = Counter()
```

It has an attribute:

```
c.value
```

and a method:

```
c.increment
```

Calling:

```
c.increment()
```

changes:

```
c.value
```

14.1 What is `self`?

When Python evaluates:

```
c.increment()
```

the object `c` is associated with the method call.

Conceptually, this is related to:

```
Counter.increment(c)
```

That is why the method definition has:

```
def increment(self):
```

`self` refers to the instance on which the method is being invoked.

Similarly:

```
self.value
```

means:

> Look up the `value` attribute on this particular instance.

This is the same dot notation we have already been using:

```
object.attribute
```

The class example simply makes the mechanism visible.

15 `__init__` and Object Construction

In the previous example:

```
c = Counter()
```

Python creates a `Counter` instance and initializes it.

The method:

```
def __init__(self, value=0):
    self.value = value
```

specifies initialization behavior.

For example:

```
c = Counter(10)

c.value
```

The key point is not to memorize the mechanics of class construction. Instead, recognize that classes can define objects with:

- attributes containing data;
- methods providing behavior;
- special methods that connect objects to Python's language-level protocols.

This is the same object model underlying Python's built-in types.

16 Objects Can Contain Other Objects

Real Python programs frequently contain nested objects.

Consider:

```
student = {
    "name": "Alice",
    "scores": [82, 91, 88]
}
```

Now:

```
student["scores"]
```

returns a list object.

That list itself has methods:

```
student["scores"].append(95)
```

We can read this expression from left to right:

```
student
   |
   v
dictionary
   |
   +-- ["scores"]
          |
          v
        list
          |
          +-- append
                 |
                 v
              method call
```

This is why Python expressions can contain multiple dots, brackets, and calls.

17 Modules Are Objects and Namespaces

Now consider:

```
import math
```

The name `math` refers to a module object.

We can inspect it:

```
type(math)
dir(math)
```

The module contains names such as:

```
math.pi
math.sqrt
math.sin
```

Thus:

```
math.sqrt(16)
```

can be understood as:

```
math
  |
  v
module object
  |
  +-- sqrt
       |
       v
    callable object
       |
       v
      (16)
```

17.1 Modules provide namespaces

A **namespace** is a mapping between names and objects.

A module therefore gives us a convenient namespace:

```
math.pi
math.sqrt
```

rather than placing every name from `math` directly into the global namespace.

This is one reason the following is generally preferable:

```
import math

math.sqrt(25)
```

to:

```
from math import *
```

The first makes the origin of `sqrt` explicit.

18 Packages and Nested Names

The same idea extends to packages and submodules.

For example:

```
import numpy as np
```

Now:

```
np.array
np.mean
np.linalg
```

and potentially:

```
np.linalg.norm
```

The expression

```
np.linalg.norm(A)
```

can be viewed as a sequence of lookups:

```
np
 |
 v
NumPy namespace
 |
 +-- linalg
      |
      v
   linear-algebra namespace
      |
      +-- norm
           |
           v
        callable
```

Similarly:

```
import matplotlib.pyplot as plt

plt.plot(x, y)
```

uses a module object and attribute access.

This object/namespace model will be especially useful when working with scientific Python libraries.

19 Namespaces and Scope

There are several places where Python looks for names.

Consider:

```
x = 10

def f():
    y = 20
    print(x)
    print(y)
```

Here `x` and `y` do not live in the same namespace.

You can inspect the current global namespace with:

```
globals()
```

and the current local namespace inside a function with:

```
def f():
    y = 20
    print(locals())
```

At the interactive prompt:

```
locals()
```

will generally show the current local namespace.

The exact rules for name lookup are described by Python's **LEGB** model:

```
L - Local
E - Enclosing
G - Global
B - Built-in
```

When Python encounters a name, it searches these scopes in this order.

For example:

```
x = "global"

def f():
    x = "local"
    print(x)

f()
```

prints:

```
local
```

because the local binding is found first.

20 Four Things That Look Similar but Are Different

Students often use the word "variable" for several different things.

Compare:

```
x
obj.x
d["x"]
```

These represent different mechanisms.

20.0.1 `x`

A name lookup.

20.0.2 `obj.x`

An attribute lookup.

20.0.3 `d["x"]`

An indexing operation on the object `d`, using `"x"` as a key.

This distinction is important.

For example:

```
student.name
```

is not the same operation as:

```
student["name"]
```

The first accesses an attribute. The second performs indexing.

A good rule is:

> **Dot notation means attribute access. Square brackets mean indexing or subscription. Neither is synonymous with "variable lookup."**

21 Introspection: Investigating an Object

Python provides powerful tools for discovering what an object is and what it can do.

21.1 `type()`

```
type(x)
```

asks:

> What type of object is this?

21.2 `id()`

```
id(x)
```

asks for the object's identity.

21.3 `dir()`

```
dir(x)
```

asks:

What names can I potentially access on this object?

21.4 `help()`

```
help(x)
help(str.upper)
```

provides documentation.

21.5 `callable()`

```
callable(x)
```

asks whether an object can be called with `()`.

These tools are particularly useful when learning a new library.

22 Looking at an Object's Attributes

For some objects, `vars()` is useful:

```
vars(obj)
```

It can show an object's attribute dictionary when the object has one.

For example, with our `Counter`:

```
c = Counter(10)

vars(c)
```

may show:

```
{'value': 10}
```

This makes the relationship between:

```
c.value
```

and the object's stored attributes more concrete.

Not every object has a `__dict__`, so `vars()` does not work universally.

23 `inspect`: Looking More Closely

The standard-library `inspect` module provides additional tools:

```
import inspect
```

For functions, you can inspect their signatures:

```
inspect.signature(sorted)
```

or:

```
inspect.signature(math.sqrt)
```

When source code is available, you can sometimes inspect it:

```
inspect.getsource(function_name)
```

This is particularly useful when exploring unfamiliar code.

The broader lesson is:

Python is unusually introspectable: programs can often examine the objects, functions, and namespaces with which they are working.

24 Reading Python Expressions Through the Object Model

The object model becomes most useful when expressions get complicated.

Consider:

```
df["return"].dropna().mean()
```

A useful way to read this is:

```
df
 ↓
DataFrame object
 ↓
["return"]
 ↓
Series object
 ↓
.dropna()
 ↓
new Series object
 ↓
.mean()
 ↓
numeric result
```

Or:

```
np.linalg.norm(A)
```

```
np
 ↓
NumPy module
 ↓
linalg
 ↓
norm
 ↓
function call with A
 ↓
result
```

The expression is not magic. It is a sequence of ordinary operations:

- name lookup;
- attribute lookup;
- indexing;
- function/method calls;
- operators.

This is an extremely useful way to approach unfamiliar code.

25 A General Strategy for Reading Python

When you encounter an unfamiliar expression, work from the outside and inside as needed.

For:

```
object.method(arg1, arg2)
```

ask:

1. What object does `object` refer to?
2. What is its type?
3. What attribute is being requested?
4. Is that attribute callable?
5. What arguments are being supplied?
6. What does the method return?
7. Does it mutate the original object?

For:

```
object.attribute
```

ask:

1. What is `object`?
2. What is its type?
3. What is the attribute?
4. Is it data or behavior?

For:

```
function(object)
```

ask:

1. What function is being called?
2. What type of object is being supplied?
3. What interface or protocol does the function expect?
4. What does it return?

This method of reading code is often more useful than memorizing syntax.

26 Worked Investigation

Suppose you encounter:

```
import statistics

data = [2, 4, 5, 7, 10]

statistics.mean(data)
```

We can investigate systematically.

26.0.1 What is `statistics`?

```
type(statistics)
```

It is a module object.

26.0.2 What does it contain?

```
dir(statistics)
```

26.0.3 What is `mean`?

```
statistics.mean
```

and:

```
callable(statistics.mean)
```

26.0.4 What arguments does it accept?

```
import inspect

inspect.signature(statistics.mean)
```

26.0.5 What happens when we call it?

```
result = statistics.mean(data)

type(result)
result
```

The general strategy is:

```
identify → inspect → understand interface → call → inspect result
```

This is a powerful approach when working with unfamiliar libraries.

27 Capstone: What Is Python Doing?

Consider:

```
class Portfolio:
    def __init__(self, values):
        self.values = values

    def mean(self):
        return sum(self.values) / len(self.values)
```

Create an object:

```
p = Portfolio([0.01, -0.02, 0.015])
```

Now investigate:

```
type(p)
dir(p)
vars(p)
```

What is:

```
p.values
```

What is:

```
p.mean
```

Is it callable?

```
callable(p.mean)
```

Now evaluate:

```
p.mean()
```

Try to explain the expression without treating it as a single indivisible operation.

A useful conceptual description is:

```
p
 ↓
Portfolio instance
 ↓
look up "mean"
 ↓
obtain method associated with p
 ↓
call method
 ↓
inside mean():
    look up p.values
    call sum(...)
    call len(...)
    divide
 ↓
return result
```

The purpose of this exercise is not to make you an expert in classes. It is to show that the same object model explains:

```
p.mean()
```

```
x.append(4)
```

```
math.sqrt(16)
```

```
np.linalg.norm(A)
```

and many other expressions you will encounter.

28 The Big Picture

We can now assemble the pieces.

28.1 Names refer to objects

```
x = [1, 2, 3]
```

binds the name `x` to a list object.

28.2 Objects have types

```
type(x)
```

tells us what kind of object `x` refers to.

28.3 Objects have attributes

```
x.attribute
```

performs attribute lookup.

28.4 Some attributes are methods

```
x.append(4)
```

looks up `append` and calls the resulting method.

28.5 Functions are callable objects

```
len(x)
```

calls the function `len` with `x` as an argument.

28.6 Special methods implement protocols

```
len(x)
x[0]
x + y
```

use standard Python interfaces associated with special methods such as:

```
__len__
__getitem__
__add__
```

28.7 Objects can be instances of classes

```
c = Counter()
```

creates an instance whose attributes and methods are defined by `Counter`.

28.8 Modules provide namespaces

```
import math

math.sqrt(16)
```

binds `math` to a module object and accesses an attribute within that namespace.

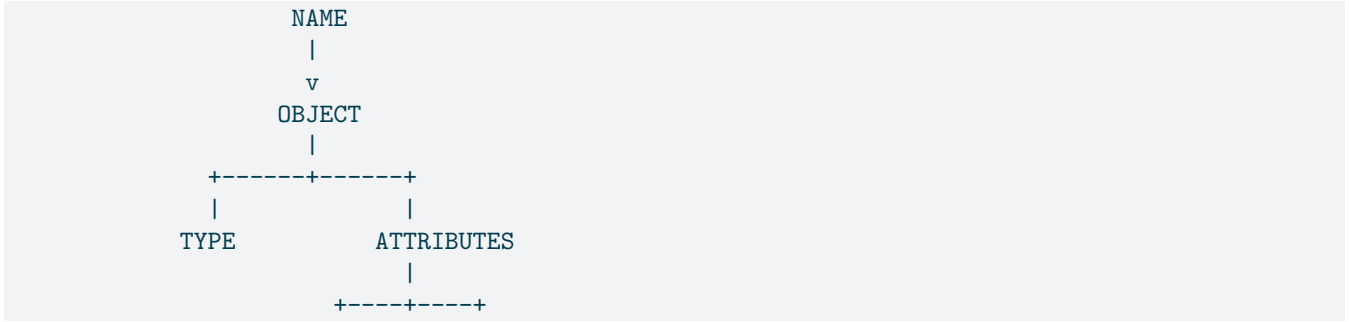
28.9 Python can inspect itself

```
type()
dir()
help()
globals()
locals()
vars()
inspect
```

provide ways to investigate objects, names, and code.

29 The Most Useful Mental Model

When reading Python, think in terms of:



```
          |         |
        DATA     METHODS
                    |
                    v
                  CALL
```

And remember the distinction between:

```
x
```

name lookup

```
x.value
```

attribute lookup

```
x[0]
```

indexing/subscription

```
x.method()
```

attribute lookup + call

```
f(x)
```

function call

```
x + y
```

language-level operation using an appropriate protocol

Once these distinctions become natural, much of Python's syntax stops looking arbitrary.

30 What You Should Take Away

You do not need to memorize every dunder method, every attribute, or every detail of Python's object model.

You should instead be comfortable with these ideas:

1. **Names refer to objects.**
2. **Every ordinary Python value is an object with a type.**
3. **The dot performs attribute lookup.**
4. **Attributes can contain data or callable behavior.**
5. **Methods are callable behavior accessed through objects.**
6. **Functions are callable objects that can be used independently of a particular object.**
7. **Python's special methods connect objects to language-level operations and protocols.**
8. **Objects can be mutable or immutable.**
9. **Classes define types and the behavior of their instances.**
10. **`self` refers to the instance on which a method operates.**
11. **Modules are objects that provide useful namespaces.**
12. **`dir()`, `type()`, `help()`, `vars()`, and `inspect` let you investigate unfamiliar code.**
13. **Complicated Python expressions can be understood as sequences of lookups, operations, and calls.**

The overarching lesson is:

> **Python syntax is not a collection of arbitrary rules. It is a language for interacting with objects through names, attributes, functions, methods, operators, and protocols.**

Once you have this model, encountering an unfamiliar Python object becomes much less intimidating: **identify it, inspect it, discover its interface, and then use that interface.**